

Git 入門

基本概念イントロダクション — Git と GitHub の違いから、日々のバージョン管理まで

対象: GitHub Copilot 試験導入プロジェクト 立上げチーム／開発部門の新任メンバー

はじめに

本資料は、社内システム開発部門で「開発ファイルのバージョン管理」を担う方が、最初に押さえておくべき Git の考え方をまとめたイントロダクションです。GitHub Copilot の試験導入では、Copilot が提案したコードも最終的には Git で履歴管理され、GitHub 上でレビュー・承認されます。つまり Git／GitHub の理解は、Copilot を安全に運用するための土台になります。

法務・AIリテラシー業務の経験から見ると、Git は次のように捉えると理解しやすくなります。

法務・文書管理の感覚	Git での対応概念
契約書の版管理（第1版→第2版→最終版）	コミット（いつ・誰が・何を・なぜ変えたかの記録）
「変更履歴の記録」機能・赤入れ	差分（diff）：行単位で変更前後を比較
ドラフトを別ファイルで並行検討	ブランチ：本流に影響を与えない作業用の分岐
決裁・稟議（承認後に正本へ反映）	プルリクエスト＋レビュー → マージ
正本の保管庫・キャビネット	リポジトリ（ローカル／リモート）
監査証跡	コミット履歴・レビュー記録（改ざんが検知可能な構造）

1. Git と GitHub の違い(比較表)

一言でいうと、Git は「バージョン管理の仕組み(ツール)」、GitHub は「Git リポジトリを預けてチームで共同作業するためのクラウドサービス」です。GitHub がなくても Git は使えますが、GitHub は Git を前提にしたサービスです。

比較項目	Git	GitHub
正体	分散型バージョン管理システム(ソフトウェア)	Git リポジトリのホスティング+共同開発プラットフォーム(Web サービス)
提供元	オープンソース(2005年 Linus Torvalds 氏が開発)	GitHub, Inc.(2018年より Microsoft 傘下)
動作場所	各自の PC(ローカル)	クラウド(github.com)※GitHub Enterprise Server は自社環境に設置可
ネット接続	不要(オフラインでコミット・履歴参照が可能)	必要
主な役割	変更履歴の記録、差分比較、ブランチ・マージ、過去版への復元	リポジトリの共有、アクセス権管理、レビュー、課題管理、自動化
操作方法	コマンドライン(git コマンド)、各種 GUI ツール、IDE	Web ブラウザ、GitHub Desktop、GitHub CLI(gh)、IDE 連携
代表的な機能	commit / branch / merge / diff / log / clone / push / pull	Pull Request、Issues、Actions(CI/CD)、Projects、Wiki、セキュリティスキャン
AI 機能	なし	GitHub Copilot(コード補完・チャット・PR 要約・コードレビュー支援など)
費用	無料(GPLv2 ライセンス)	無料プランあり。組織利用は Team/Enterprise など有料プラン、Copilot は別途ライセンス
権限・統制	基本的に持たない(ファイルにアクセスできれば操作可能)	組織・チーム単位の権限、ブランチ保護、必須レビュー、監査ログ、SSO 等
代替・類似	Subversion(SVN)、Mercurial など	GitLab、Bitbucket、Azure Repos など
例え	文書の「版管理の仕組み」そのもの	版管理された文書を保管・共有・決裁する「共有キャビネット+ワークフロー」

ポイント: Copilot 試験導入で議論する「誰がどのリポジトリにアクセスできるか」「レビュー必須にするか」「利用ログをどう残すか」といった統制は、主に GitHub 側の設定で実現します。一方、日々の変更記録そのものは Git の役割です。

2. バージョン管理とは

バージョン管理とは、ファイルの変更履歴を記録し、「いつ・誰が・何を・なぜ」変更したかを後から確認したり、過去の状態に戻したりできるようにする仕組みです。「仕様書_最終版_修正2_本当に最終.docx」のようなファイル名による管理から卒業し、1つのファイルの中で履歴を一元管理できます。

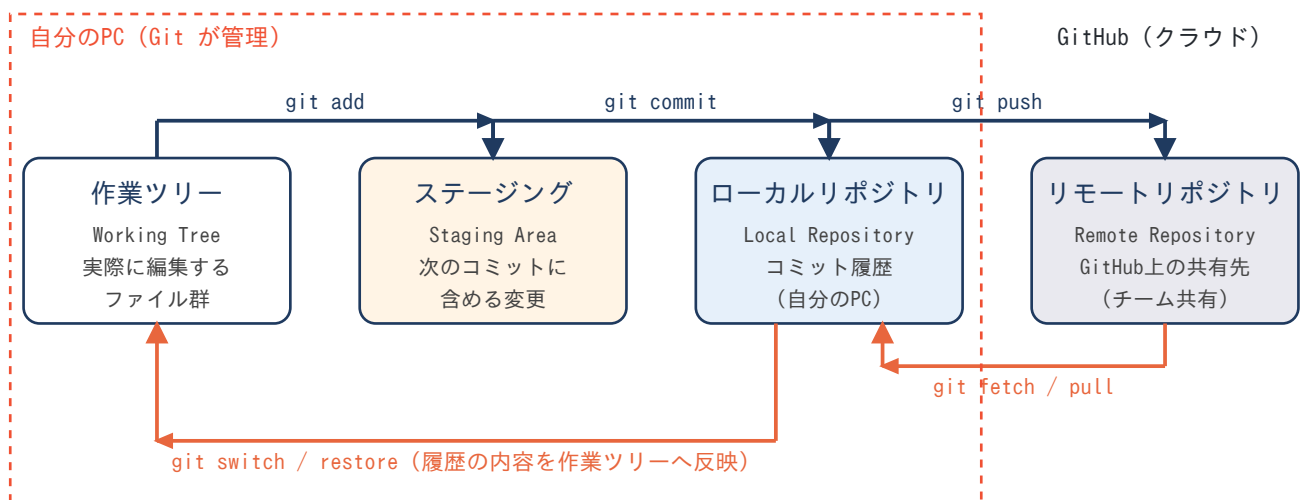
- 追跡性: 変更点と理由(コミットメッセージ) がすべて残る
- 復元性: 誤った変更をしても、任意の時点に戻せる
- 並行作業: 複数人が同じファイル群を同時に安全に編集できる
- 説明責任: レビューと承認の記録が残り、監査に対応しやすい

集中型と分散型

Subversion などの集中型は、履歴を中央サーバーだけが持ちます。Git は分散型で、各メンバーの PC に履歴を含むリポジトリの完全な複製があります。そのため、オフラインでも作業でき、サーバー障害時にも履歴が失われにくいという特長があります。

3. Git の基本構造: 4つの場所

Git を理解するうえで最も重要なのが、ファイルの変更が次の4つの場所を順に移動するというイメージです。



場所	説明
作業ツリー	普段エディタで編集している、目に見えるファイルそのもの。
ステージング(インデックス)	「次のコミットに含める変更」を選んで置いておく準備エリア。変更の一部だけを記録したいときに役立つ。
ローカルリポジトリ	コミットした履歴が保存される場所(.git フォルダ)。自分の PC 内にある。
リモートリポジトリ	GitHub などに置かれた共有用リポジトリ。push で送り、fetch/pull で取り込む。

4. 主要な概念

リポジトリ (Repository)

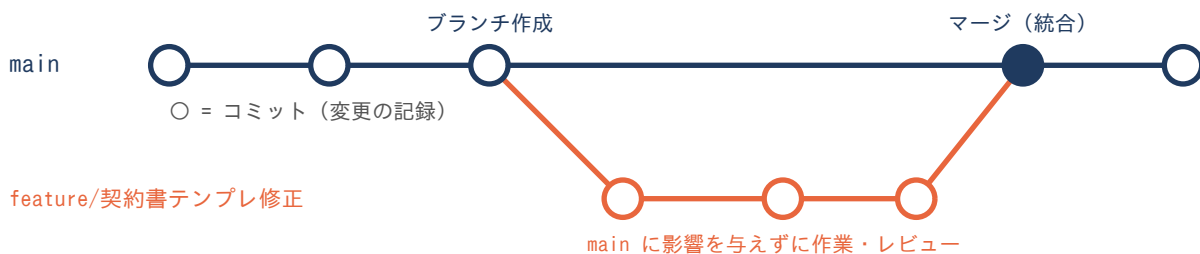
プロジェクトのファイルと、その全変更履歴をまとめて保管する入れ物です。git init で新規作成するか、git clone で GitHub から複製して始めます。

コミット (Commit)

ある時点のファイルの状態を記録した「スナップショット」です。各コミットには固有の ID (ハッシュ値。例: 58ad506)、作成者、日時、メッセージが付きます。ハッシュ値は内容から計算されるため、過去の履歴を書き換えると ID が変わり、改ざんが検知可能です。コミットメッセージは「何を・なぜ」を簡潔に書くのが基本です。

ブランチ (Branch) とマージ (Merge)

ブランチは、本流 (通常 main) から分岐して作業するための仕組みです。新機能や修正はブランチで行い、確認が済んだら本流へマージ (統合) します。



同じ箇所を複数人が別々に変更するとコンフリクト (競合) が発生します。Git は自動で決められないため、人が内容を確認してどちらを採用するか決めます。

リモート (Remote) と同期

- clone: リモートリポジトリを丸ごと手元に複製する (最初の1回)
- push: 自分のコミットをリモートへ送る
- fetch: リモートの最新履歴を取得する (作業ツリーは変えない)
- pull: fetch + 取り込み (マージ) をまとめて行う

プルリクエスト (Pull Request, PR) ※GitHub の機能

「このブランチの変更を main に取り込んでください」という依頼です。差分の確認、コメント、承認、自動テストの結果が1か所に集約され、社内の決裁フローに相当する記録になります。Copilot が生成したコードも、PR でのレビューを経てから本流に入れる運用が基本です。

5. よく使う Git コマンド

目的	コマンド	説明
初期設定	git config --global user.name "氏名" git config --global user.email "メール"	コミットに記録される作成者情報を設定（最初に1回）
開始	git init / git clone <URL>	新規リポジトリ作成 / GitHub から複製
状態確認	git status	変更・ステージング状況を表示（迷ったらまずこれ）
差分確認	git diff	変更内容を行単位で表示
ステージ	git add <ファイル>	次のコミットに含める変更を選ぶ
記録	git commit -m "メッセージ"	ステージした変更を履歴に記録
履歴	git log --oneline	コミット履歴を一覧表示
ブランチ	git branch / git switch -c <名前>	ブランチ一覧 / 新規作成して切替
統合	git merge <ブランチ>	指定ブランチの変更を現在のブランチへ統合
同期	git push / git pull / git fetch	リモートへ送信 / 取得して統合 / 取得のみ
取り消し	git restore <ファイル> git revert <コミットID>	作業中の変更を破棄 / 過去のコミットを打ち消す新コミットを作成

注意: git push --force や git reset --hard は履歴や作業内容を消す可能性がある操作です。共有ブランチでは使用しない、または GitHub の「ブランチ保護ルール」で禁止するのが一般的です。

6. 基本的な作業の流れ (GitHub Flow)

手順	操作	ポイント
1	main を最新化 (git switch main → git pull)	古い状態から作業を始めない
2	作業ブランチを作成 (git switch -c feature/xxx)	1つの目的に1ブランチ
3	編集 → git add → git commit を繰り返す	小さく、意味のある単位でコミット
4	git push でブランチを GitHub へ送る	バックアップ兼共有
5	GitHub でプルリクエストを作成	目的・変更点・確認方法を記載
6	レビュー・自動テスト → 修正	Copilot 生成コードも人が必ず確認
7	承認後に main へマージ、ブランチを削除	記録は PR に残る

7. GitHub Copilot 試験導入に向けた留意点

これまでの法務・AI リテラシーの知見が特に活きる領域です。導入前に、以下を Git／GitHub の運用ルールとして整理しておくことを推奨します。

観点	確認・検討事項
機密情報の管理	パスワード・API キー・個人情報をリポジトリにコミットしない。 <code>.gitignore</code> で除外し、GitHub のシークレットスキャンを有効化する。Git の履歴は残り続けるため、一度コミットした秘密情報は「削除」ではなく無効化（キーの再発行）が必要。
Copilot の参照範囲	Copilot Business／Enterprise の「コンテンツ除外」設定で、機密性の高いファイルやリポジトリを Copilot の参照対象から外せるか検討する。
ライセンス・著作権	公開コードと一致する提案をブロックする設定（パブリックコードとの一致の抑制）の有効化を検討。利用する OSS のライセンス表記の扱いをルール化する。
データの取扱い	プロンプトや提案データの保存・学習利用の有無は、契約プランと GitHub の最新の利用規約・プライバシー文書で確認する。
レビュー体制	ブランチ保護で main への直接 push を禁止し、PR レビュー承認を必須化。「AI が書いたコードも人が責任を持って承認する」原則を明文化する。
アクセス権	組織・チーム単位で最小権限を付与。退職・異動時の権限削除手順を定める。
記録・監査	コミット履歴・PR・監査ログにより「誰が何を承認したか」を追跡可能にする。Copilot の利用状況は組織の管理画面・監査ログで確認できる範囲を把握する。

※ Copilot の機能名・設定項目・プランの内容は更新が頻繁なため、導入判断の際は GitHub 公式ドキュメントと契約条件の最新版を必ず確認してください。

8. 用語集

用語	意味
リポジトリ	ファイルと変更履歴の保管場所
コミット	変更の記録(スナップショット)
ブランチ	本流から分岐した作業ライン
マージ	ブランチの変更を統合すること
コンフリクト	同じ箇所への異なる変更が衝突した状態
クローン	リモートリポジトリの複製を手元に作ること
プッシュ／プル	リモートへの送信／リモートからの取得と統合
プルリクエスト	変更の取り込みを依頼し、レビューを受ける GitHub の機能
HEAD	現在作業している位置(コミット)を指す目印
.gitignore	Git の管理対象から除外するファイルを指定する設定ファイル

9. 次のステップ

- テスト用リポジトリを作成し、clone → 編集 → commit → push → PR → マージを一通り体験する
- ブランチ保護・必須レビューなどの GitHub 組織設定を、試験導入チームで確認する
- Copilot 利用ガイドライン(機密情報・ライセンス・レビュー責任)の草案を作成する
- 参考: Pro Git (日本語版・無料公開) <https://git-scm.com/book/ja/v2> / GitHub Docs <https://docs.github.com/ja>